

04. Divide-and-Conquer

Hyunjong Choi, Ph.D.

Assistant Professor of Computer Science

San Diego State University

Outline

❑ More algorithms based on *divide-and-conquer*

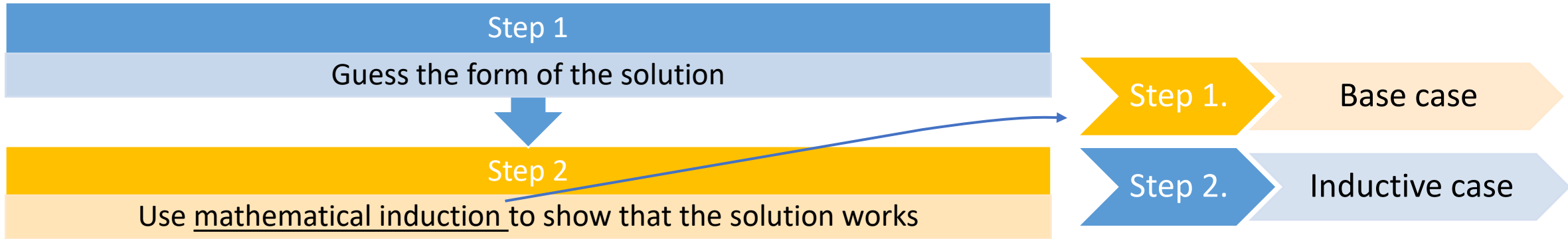
- Maximum-subarray problem (Textbook 3rd edition 68 – 74)
- Matrix multiplication problem (straightforward method vs. Strassen's algorithm) (Textbook 3rd edition 75 – 82)

❑ Solving recurrence equations

- ***Substitution method*** for solving recurrence (Textbook 3rd edition 83 – 87)
- ***Recursion-tree method*** (Textbook 3rd edition 88 – 92)
- Master method (master theorem) (Textbook 3rd edition 93 – 106)

Substitution method

- ❑ The most robust method of solving recurrences



- ❑ “**Substitution**”: substitute the guessed solution for the function when applying the inductive hypothesis to smaller values

$$T(n) = \begin{cases} 1, & \text{if } n = 1 \\ 2T(\lfloor n/2 \rfloor) + n, & \text{if } n > 1 \end{cases}$$

A blue thought bubble containing the text: $T(n) = O(n \cdot \lg n)$
 $\leftrightarrow T(n) \leq cn \cdot \lg n$

Our guess

➡ Need a good guess first

Mathematical induction (example)

□ **Problem:** Prove that $3^n - 1$ is a multiple of 2

- **Step 1**, find a boundary case:

✓ For $n = 1$, $3^1 - 1 = 2$, which is a multiple of 2 $\Rightarrow$ True!

- **Step 2**, assume the claim (inductive hypothesis) is true for $n = k$, i.e., " $3^k - 1$ is a multiple of 2"

✓ Then, we need to prove that " $3^{k+1} - 1$ is also a multiple of 2".

$$\checkmark 3^{k+1} - 1 = 3 \times 3^k - 1 = 2 \times 3^k + (3^k - 1)$$

Multiple of 2

Multiple of 2

The sum is also a multiple of 2. (Proof is done)

➡ The claims holds for all integer numbers (e.g., $n > 0$)

Substitution method: example

□ Consider a recurrence: $T(n) = \begin{cases} 1, & \text{if } n = 1 \\ 2T(\lfloor n/2 \rfloor) + n, & \text{if } n > 1 \end{cases}$, which is similar to the maximum subarray algorithm and merge sort.



Step 1. Make a guess: $T(n) = O(n \cdot \lg n)$

Step 2. Need to prove that $T(n) \leq cn \cdot \lg n$ for a constant $c > 0$, and for $n > n_0$

- Inductive hypothesis
 - ✓ Start by **assuming** that this bound holds for all positive $m < n$, in particular $m = \lfloor n/2 \rfloor$
 - ✓ Inductive hypothesis yields $T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)$
- Substitute this into the recurrence equation ...

Substitution method: example (cont.)

□ Proof by mathematical induction

- **Step 2. Inductive step:** if we **substitute** the inductive hypothesis into recurrence

$$T(n) \leq 2(c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \longrightarrow \bullet \text{ Remove floors}$$

$$\leq 2(c(n/2) \lg(n/2)) + n$$

$$= cn \lg(n) - cn \lg 2 + n$$

$$= cn \lg(n) + (1 - c)n \leq \mathbf{cn \lg(n)}$$

- This inequality holds as long as $c \geq 1$

- We have proven that if $T(\lfloor n/2 \rfloor) \leq c(\lfloor n/2 \rfloor) \lg(\lfloor n/2 \rfloor)$ holds, then $T(n) \leq cn \lg n$ must hold (as long as $c \geq 1$)

 We have done the inductive step.

- **Step 1. Base case:** now, prove that $T(n) \leq cn \lg n$ holds for some specific **boundary** conditions, i.e., the starting point of the induction.

✓ What about the base-case, $n = 1$?

✓ However, $T(1) = 1 > c1 \cdot \lg 1$, which violates the solution

Substitution method: example (cont.)

- Let's consider $n = 2$ and $n = 3$ as the boundary conditions
 - ✓ $T(2) = 2T(1) + 2 = 4$, $T(3) = 2T(1) + 3 = 5$
 - ✓ As long as $T(2) = 4 \leq c \cdot 2 \cdot \lg 2$ and $T(3) = 5 \leq c \cdot 3 \cdot \lg 3$ hold, the boundary is proven
 - ✓ Solving the inequalities, we have $c \geq 2$.

□ Altogether, $T(n) \leq cn \lg n$ always holds with $c \geq 2$ and $n \geq n_0 = 2$ (smallest boundary condition)

➡ Therefore, $T(n) = O(n \lg n)$. Done!

Tips for using substitution method

- ❑ Unfortunately, there is no general way to guess the correct solutions to recurrences.
 - Use some heuristics, e.g., recursion trees, to help guess
- ❑ Tip 1. If a recurrence is similar to one we met before, then guessing a similar solution.
 - e.g., $T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$
 - When n is large, $\lfloor n/2 \rfloor + 17$ is not that different from $\lfloor n/2 \rfloor$
 - We can make the guess that $T(n) = O(n \lg n)$.
- ❑ Tip 2. Determine loose upper bounds on the recurrence, then reduce your range of uncertainty.
 - e.g., start with a loose lower bound $T(n) = O(n^2)$.
 - Then, trying to lower the upper bound until the solution becomes more accurately.

Outline

❑ More algorithms based on *divide-and-conquer*

- Maximum-subarray problem (Textbook 3rd edition 68 – 74)
- Matrix multiplication problem (straightforward method vs. Strassen's algorithm) (Textbook 3rd edition 75 – 82)

❑ Solving recurrence equations

- *Substitution method* for solving recurrence (Textbook 3rd edition 83 – 87)
- ***Recursion-tree method*** (Textbook 3rd edition 88 – 92)
- Master method (master theorem) (Textbook 3rd edition 93 – 106)

Recursion-tree method

❑ Two usages of recursion-tree method

Use case 1

- Devise a good guess
- Then, verify by substitution method
- Tolerate a small amount of sloppiness

Use case 2

- Solve recurrence directly
- e.g., merge sort

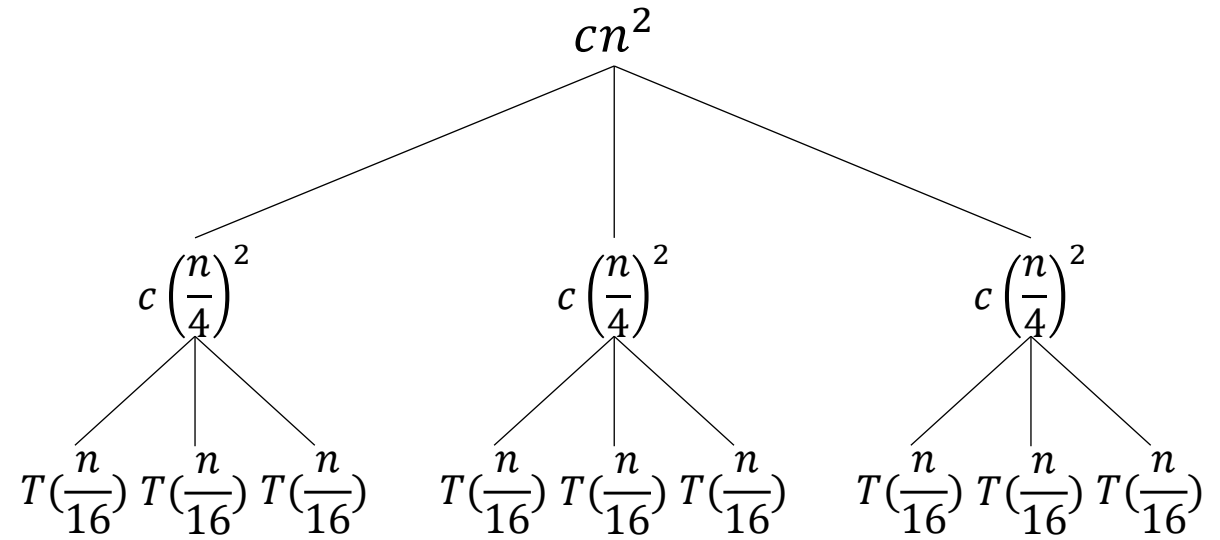
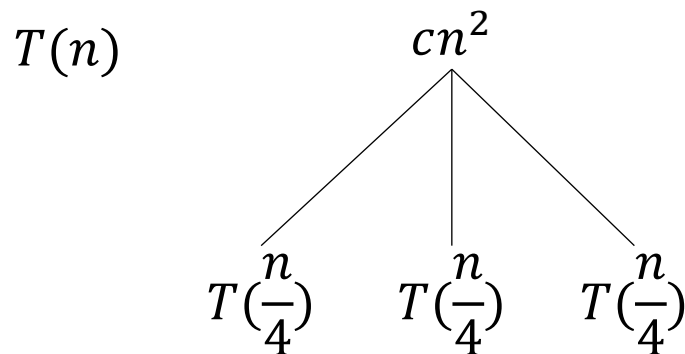
Example 1 : Recursion-tree method

Use case 1

□ Given $T(n) = \begin{cases} 1 & , n = 1 \\ 3T(\lfloor n/4 \rfloor) + \Theta(n^2) & , n > 1 \end{cases}$, what's the upper bound of $T(n)$?

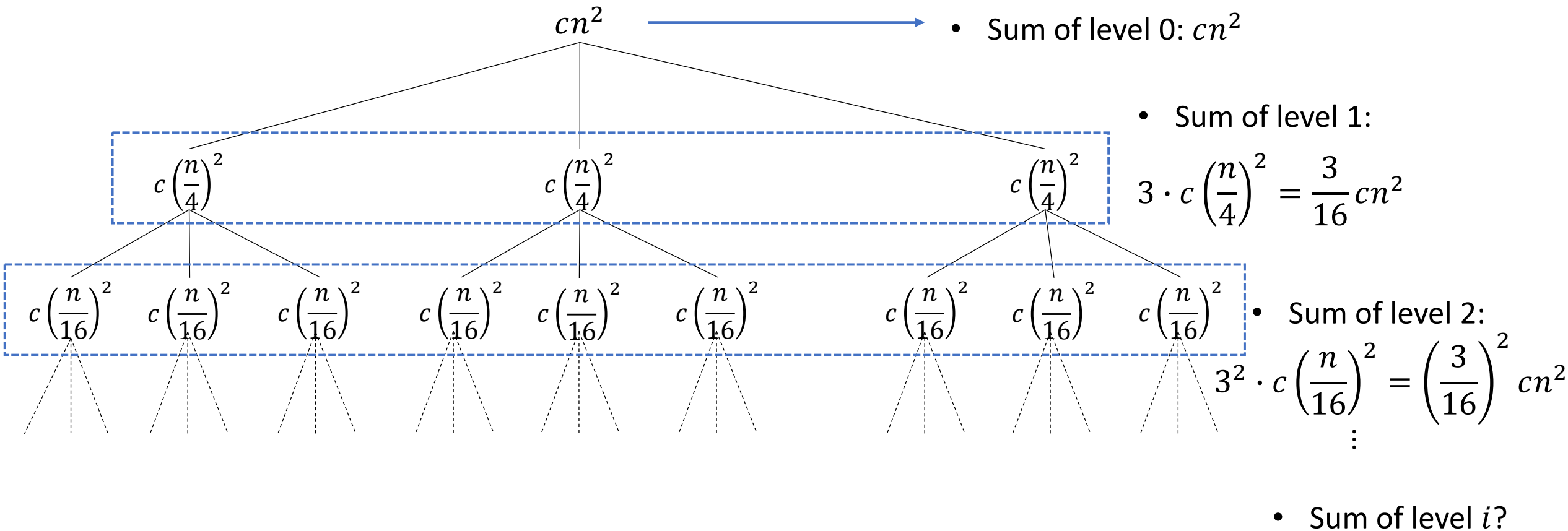
- Use recursion-tree for a good guess
- Floor function doesn't affect to draw tree (sloppiness)

➡ Draw a recursion tree for $T(n) = 3T(n/4) + cn^2$



Recursion-tree: Expanding tree

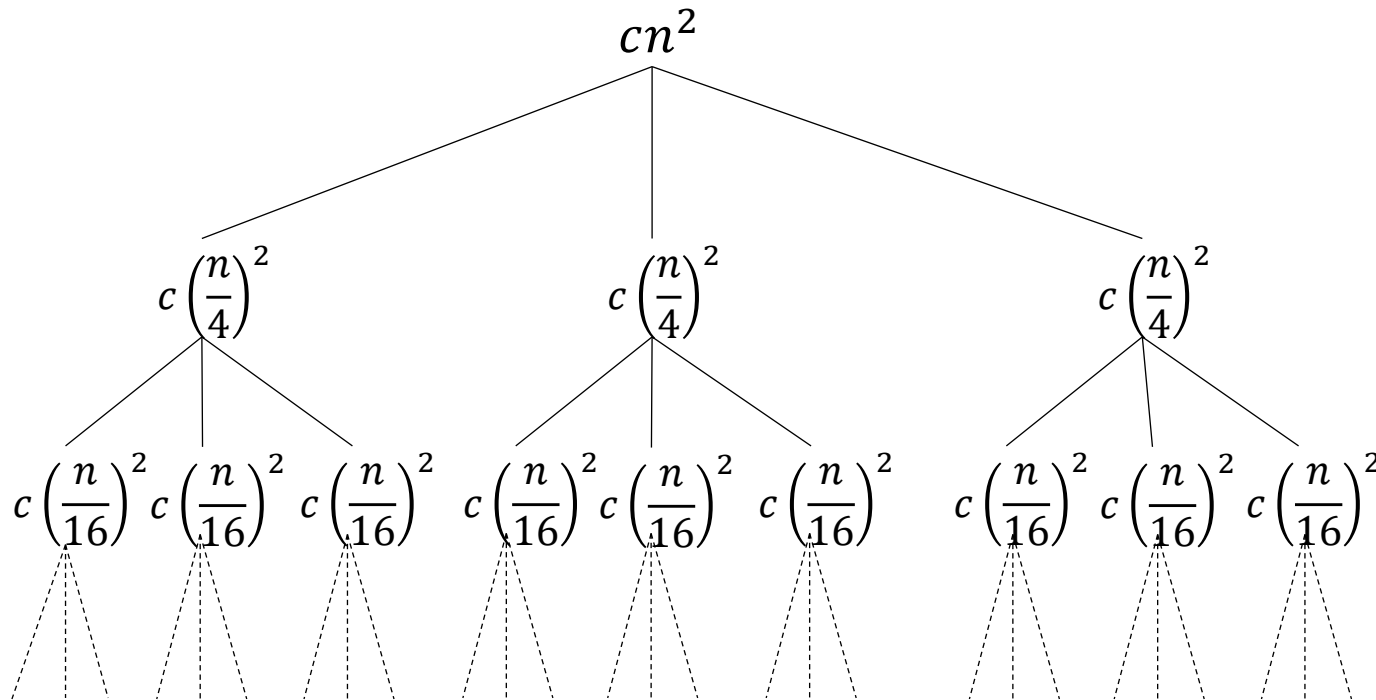
Use case 1



Recursion-tree: Depth of the tree

Use case 1

- When does the expanding end?
- What's the number of levels ?



- Subproblem sizes decrease by a factor of 4:

$$T(n) \rightarrow T\left(\frac{n}{4}\right) \rightarrow T\left(\frac{n}{4^2}\right) \rightarrow T\left(\frac{n}{4^3}\right) \dots$$

- The subproblem size at depth i is $\frac{n}{4^i}$
- The subproblem size hits 1 at the leaf nodes

$$\frac{n}{4^i} = 1 \quad \text{i.e., } i = \log_4 n$$

- Thus, the tree has $\log_4 n + 1$ levels (at depth $0, 1, 2, \dots, \log_4 n$)

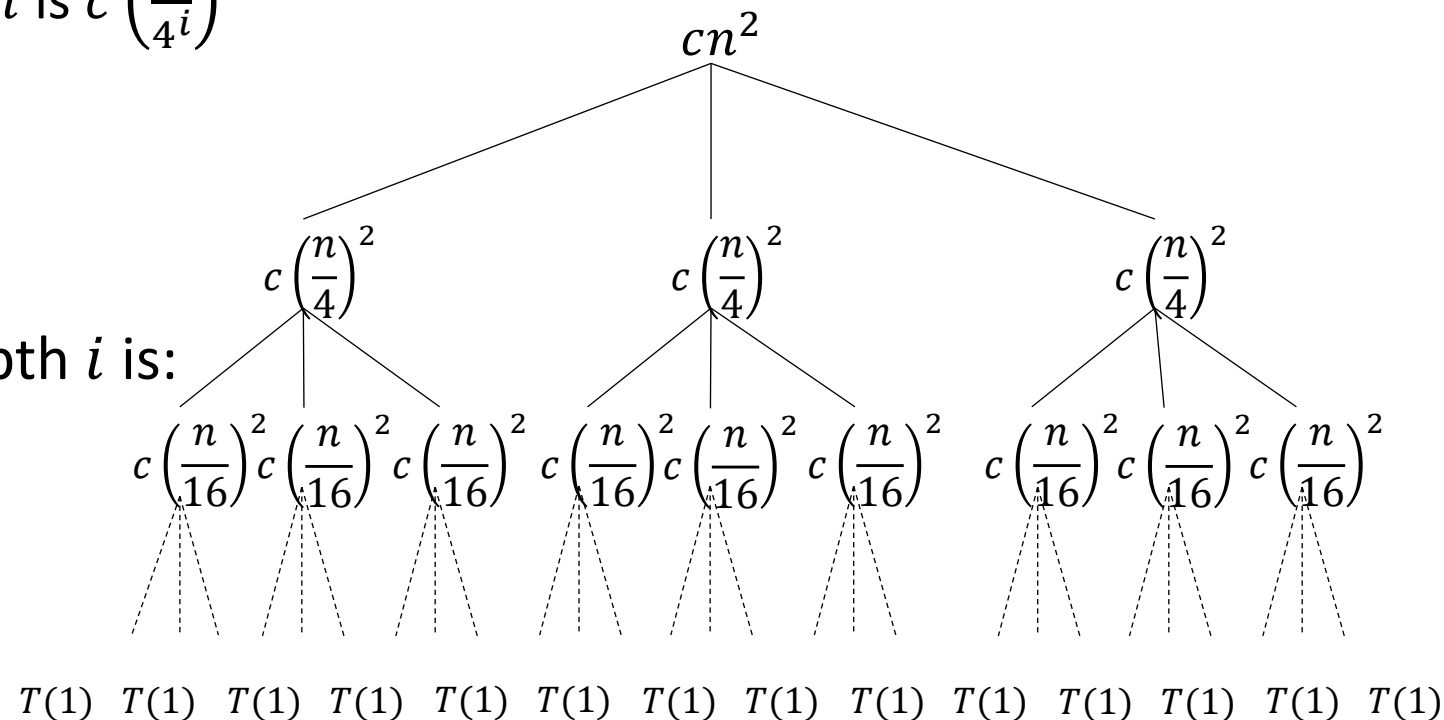
Recursion-tree: Costs per level

- ❑ Each level has three times more nodes than the level above
 - The number of nodes at depth i is 3^i
- ❑ The subproblem sizes reduce by a factor of 4 for each level
 - The cost for each node at depth i is $c \left(\frac{n}{4^i} \right)^2$



- ❑ The total cost for the level at depth i is:

$$3^i \cdot c \left(\frac{n}{4^i} \right)^2 = \left(\frac{3}{16} \right)^i cn^2$$



Recursion-tree: Bottom level cost

Use case 1

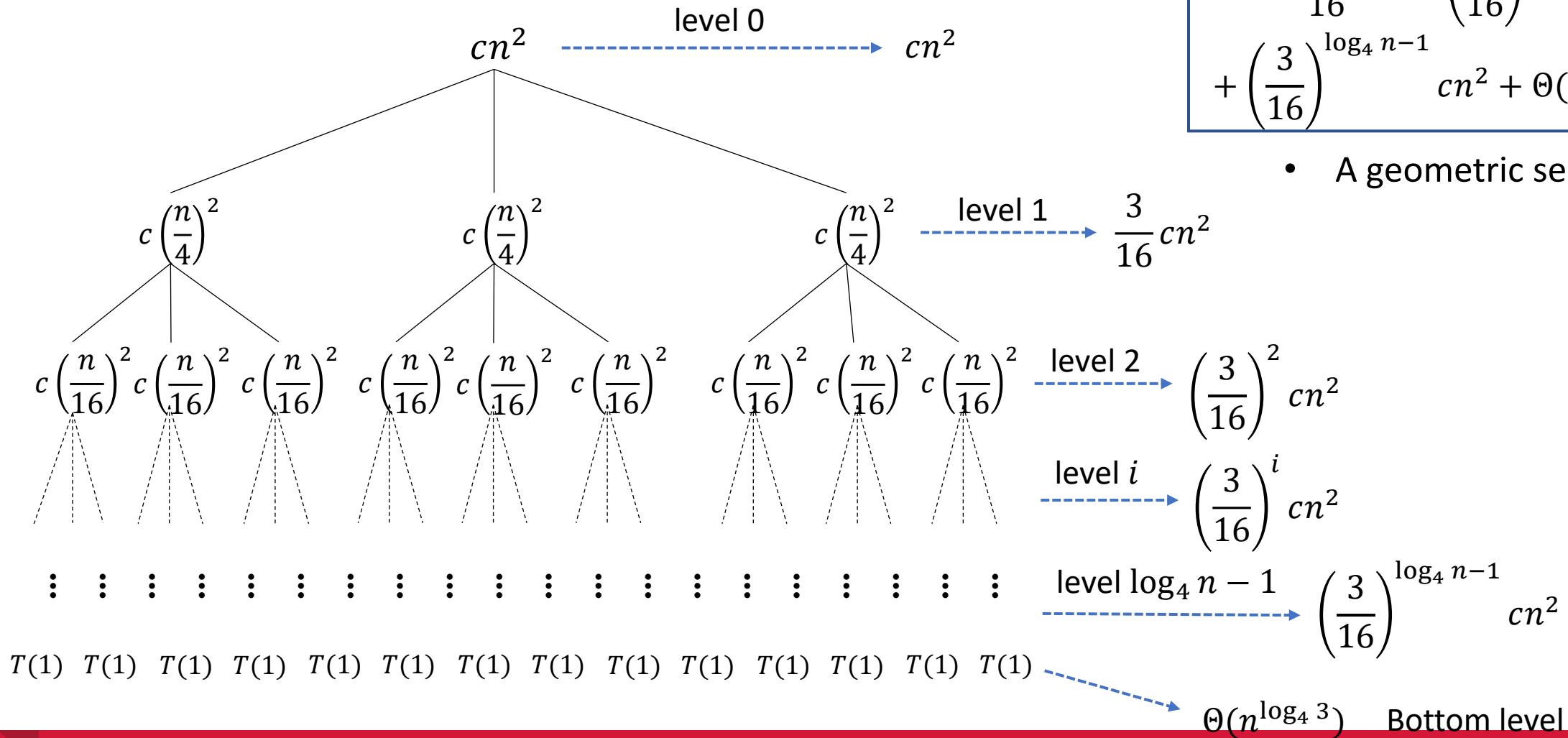
□ Get the bottom-level (leaf) cost

- Bottom level is at the depth $i = \log_4 n$
- The number of leaf nodes: $3^i = 3^{\log_4 n} = n^{\log_4 3}$
- Each leaf node has a cost of $T(1)$

➡ The total cost for bottom level is: $T(1)n^{\log_4 3} = \Theta(n^{\log_4 3})$

Recursion-tree: Total cost

Use case 1



Recursion-tree: Total cost (cont.)

Use case 1

□ Summation of a geometric series

$$\begin{aligned} T(n) &= cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3}) \\ &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\ &= \frac{\left(\frac{3}{16}\right)^{\log_4 n} - 1}{\frac{3}{16} - 1} cn^2 + \Theta(n^{\log_4 3}) \end{aligned}$$

Recursion-tree: Total cost (cont.)

Use case 1

□ Use the property of geometric series

$$T(n) = \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3})$$

$$< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3})$$

$$= \frac{1}{1 - \frac{3}{16}} cn^2 + \Theta(n^{\log_4 3}) = \frac{16}{13} cn^2 + \Theta(n^{\log_4 3})$$

$$= O(n^2)$$

- Now, we have derived a guess $T(n) = O(n^2)$ for the original recurrence:
- $T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$

≈ 0.792

Using substitution method

□ We want to prove $T(n) = O(n^2)$, i.e., there exists some constant $d > 0$, such that $T(n) \leq dn^2$ for $n > n_0$

- **Step 2. Inductive step**

$$\begin{aligned} T(n) &= 3T(\lfloor n/4 \rfloor) + n^2 \leq 3T(n/4) + cn^2 \\ &\leq 3d \left(\frac{n}{4}\right)^2 + cn^2 \\ &= \frac{3}{16}dn^2 + cn^2 \\ &\leq dn^2 \longrightarrow \text{This inequality holds as long as } d \geq \frac{16}{13}c \end{aligned}$$

⇒ The second step of mathematical induction is done.

- **Step 1. Base case**

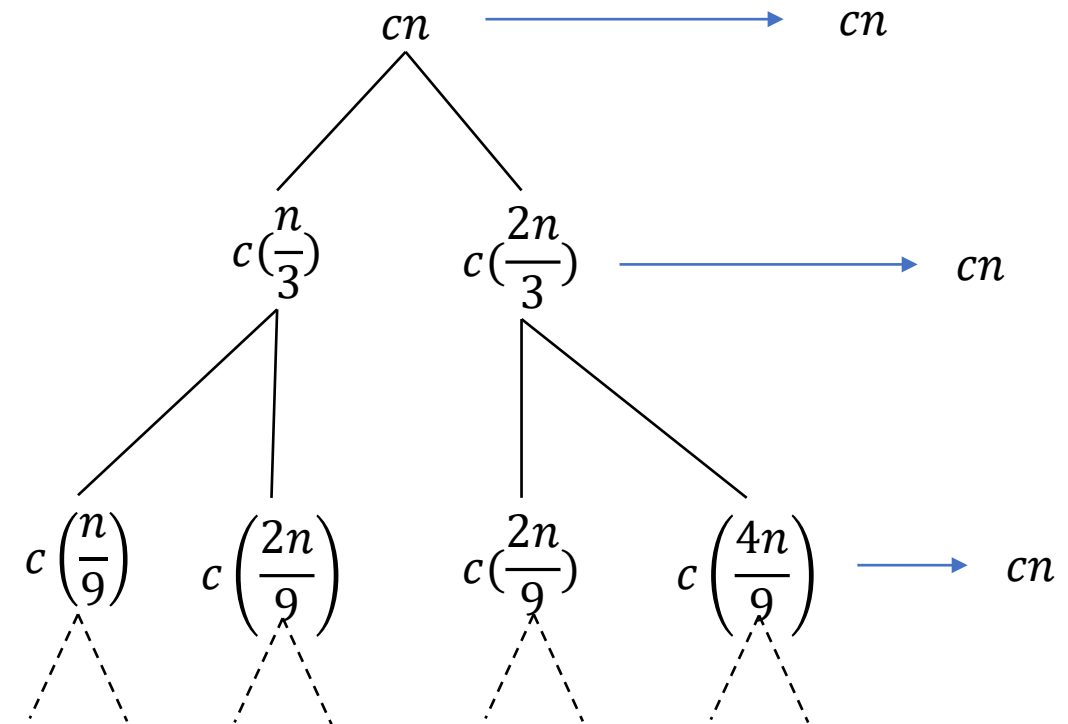
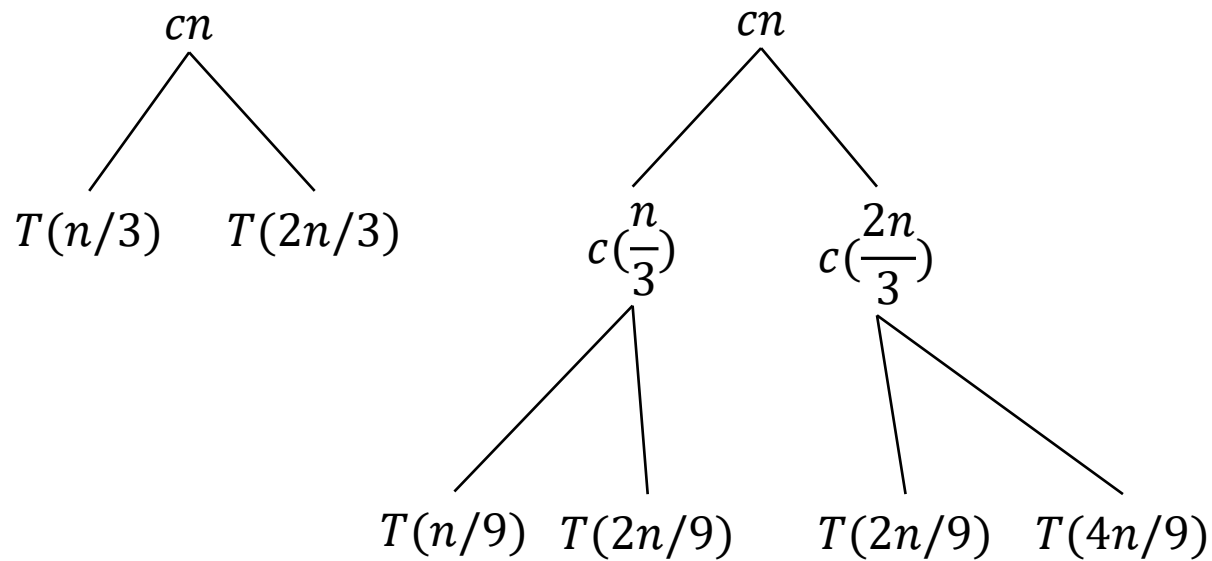
✓ A boundary condition should be checked to find a proper n_0 .

Example 2 : Recursion-tree method

Use case 1

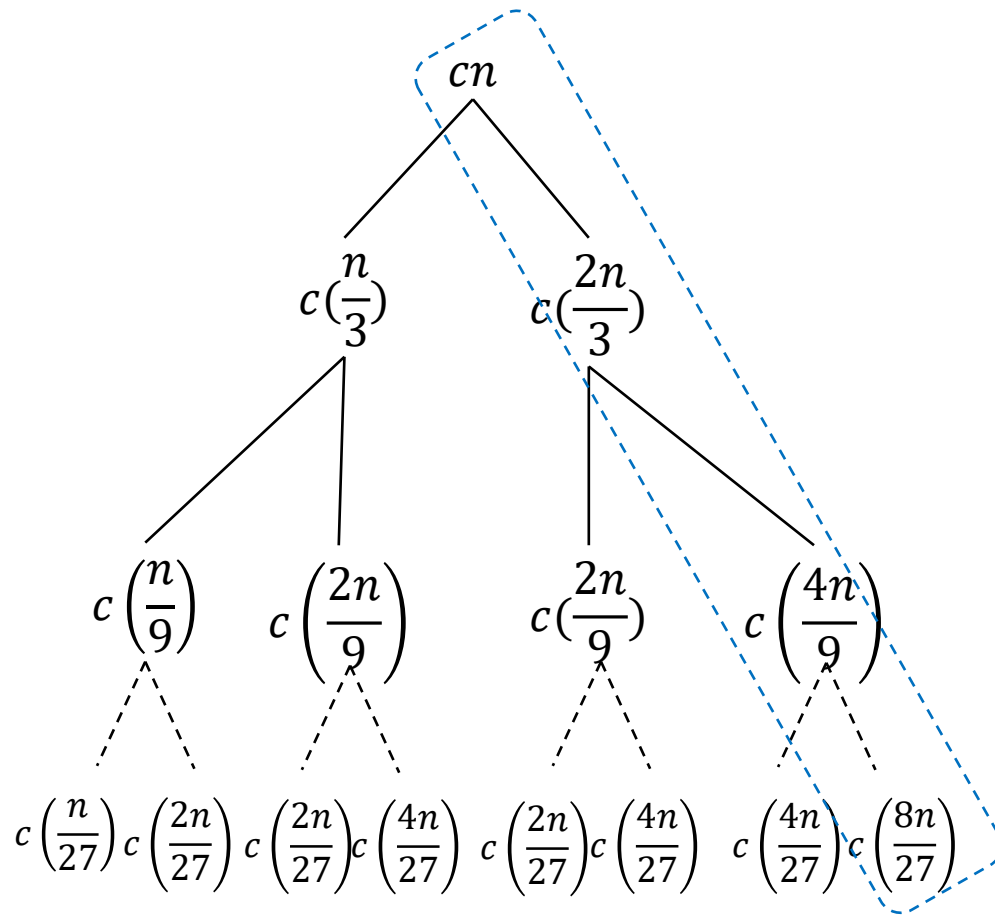
□ Unbalanced recurrence: $T(n) = T(n/3) + T(2n/3) + cn$

Per level sum:



Branches grow in different speed

Use case 1



- Some nodes hit $T(1)$ earlier than others
- Consider the longest path from root to a leaf:

$$n \rightarrow \left(\frac{2}{3}\right)n \rightarrow \left(\frac{2}{3}\right)^2 n \rightarrow \dots \left(\frac{2}{3}\right)^i n \rightarrow \dots 1$$

- When $\left(\frac{2}{3}\right)^i n = 1$, $i = \log_{3/2} n$
- Thus, the height of the tree is $\log_{3/2} n + 1$.
- **If** it is a binary tree (balanced), then the total cost is:

$$cn \cdot \log_{3/2} n = \mathbf{O}(n \lg_{3/2} n)$$

- However, this is **not** a binary tree: it has fewer than $2^{\log_{3/2} n}$ leaf nodes; *many internal nodes are absent* $\rightarrow$ the per level sum is not necessarily cn

Tolerate the sloppiness

- ❑ Remember: we are just trying to come up with a *guess* out of the recursion plot.
 - $T(n) = \mathbf{O(n \lg n)}$ could be a good guess to start with
- ❑ Use substitution method to verify it, i.e., to show that $T(n) \leq d \cdot n \lg n$, with proper choice of d

$$T(n) = T(n/3) + T(2n/3) + cn$$

$$\leq d \left(\frac{n}{3}\right) \lg \left(\frac{n}{3}\right) + d \left(\frac{2n}{3}\right) \lg \left(\frac{2n}{3}\right) + cn$$

$$= d \left(\frac{n}{3}\right) (\lg n - \lg 3) + d \left(\frac{2n}{3}\right) (\lg 2 + \lg n - \lg 3) + cn$$

$$= dn \lg n - d \left(\lg 3 - \frac{2}{3}\right) n + cn$$

$$\leq dn \lg n$$

This inequality holds as long as $d \geq \frac{c}{\lg 3 - \frac{2}{3}}$

- The second step of mathematical induction is proved!

Recap

- ☐ Drawing a recursion tree is helpful to devise a *guess* to the solution of the recurrence.
- ☐ The tree does not have to be a complete one.
- ☐ Be bold in deriving a guess.

Example

Problem

□ Q. Use the substitution method to show that the recurrence has the solution:

- $T(n) = \begin{cases} 1, & n = 1 \\ T(n-1) + n, & \text{o.w.} \end{cases}$ has solution $T(n) = O(n^2)$

Example

Problem

□ Q. Use the substitution method to show that the recurrence has the solution:

$$\bullet \quad T(n) = \begin{cases} 1, & n = 1 \\ T(n-1) + n, & \text{o.w.} \end{cases} \quad \text{has solution } T(n) = O(n^2)$$

→ We guess that $T(n) \leq cn^2$ for some constant $c > 0$.

• Step 1. assume that $T(n-1) \leq c(n-1)^2$ is true, then show $T(n) \leq cn^2$

$$\rightarrow T(n) = T(n-1) + n$$

$$\rightarrow T(n) \leq c(n-1)^2 + n$$

$$= cn^2 - 2cn + c + n \leq cn^2$$

$$= cn^2 + c(1-2n) + n$$

The last inequality holds if $c(1-2n) + n \leq 0$

i.e., $c \geq \frac{n}{2n-1}$, which holds for all $n \geq 1, c \geq 1$

• Step 2. boundary condition,

→ We set $T(1) = 1$, and so $T(1) = 1 \leq c \cdot 1^2$.

Therefore, we can choose $n_0 = 1$ and $c = 1$.